

Implementation and Performance Evaluation of a Blockchain- Anchored Log Auditing Framework

MRF Nusfa-it22908742@my.sliit.lk

Sanjula E A Y-it22340078@my.sliit.lk

Salgado M B D-it21289934@my.sliit.lk

De Silva H S-it22887580@my.sliit.lk

*Department of Information Technology
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka*

Abstract — Modern distributed systems generate massive volumes of log data that serve as the backbone of security monitoring, regulatory compliance, and forensic investigation. However, conventional storage architectures offer no cryptographic guarantees against post-hoc modification of log records. Blockchain-based anchoring presents a compelling solution: by committing cryptographic digests of log groups to an immutable ledger, any retrospective alteration becomes immediately detectable. This paper presents an end-to-end implementation of a LogStamping log auditing framework built on a private Ethereum Proof-of-Authority (Clique) network. Log entries are aggregated into configurable groups, fingerprinted with SHA-256, and exclusively the resulting digests are anchored on-chain via a Solidity smart contract, with optional archival to IPFS and indexing in Elasticsearch for efficient retrieval. Five algorithms governing ingestion, hashing, blockchain submission, smart contract logic, and integrity verification are formally specified and illustrated with architectural and flow diagrams. Empirical evaluation across 2,000 to 12,000 log entries demonstrates that at 10,000 entries, LogStamping achieves approximately 155,000 ms verification time with only 0.62 MB blockchain storage — outperforming EngraveChain (221,000 ms / 12.5 MB) and ProvChain (209,000 ms / 10.0 MB) in storage efficiency. Comparison with BCALS reveals a verification-time trade-off inherent to hash-only anchoring. The results collectively validate grouped hash anchoring as a scalable, storage-efficient strategy for production-grade log integrity assurance.

Index Terms — Blockchain, log integrity, audit trail, Proof-of-Authority, smart contracts, SHA-256, tamper detection, distributed systems, IPFS, Elasticsearch.

I. INTRODUCTION

The integrity of system logs is foundational to the trustworthiness of modern digital infrastructure. Log records generated by servers, firewalls, routers, database engines, and

application middleware collectively constitute the primary evidentiary basis for security incident response, compliance auditing, and post-breach forensic reconstruction [32][33][34]. Yet these records are stored in mutable file systems, making them vulnerable to targeted deletion or modification by adversaries seeking to obscure their activities or obstruct investigations [37].

Traditional countermeasures — centralized log servers, write-once storage media, and role-based access controls — reduce but cannot eliminate the threat surface. A centralized aggregation point introduces a single point of failure both operationally and forensically: a sufficiently privileged attacker who compromises the aggregation server can rewrite history without leaving cryptographic evidence. Cloud-based storage approaches inherit trust assumptions from the hosting provider, raising concerns about unauthorized access and data consistency [5].

Blockchain technology offers a structurally different guarantee. By anchoring cryptographic commitments to a decentralized, append-only ledger governed by consensus, even a node operator cannot retroactively alter previously committed data without invalidating the entire subsequent chain — a modification that all other participants would immediately detect [35]. Prior work has explored this property for log management [6][12][15][16][17][19], but existing systems suffer from one or more critical shortcomings: massive on-chain storage overhead that grows with both log volume and validator count, inadequate support for real-time continuous log streams, confidentiality failures when raw payloads are stored on-chain, and absence of production-grade performance characterization.

This paper addresses these gaps through a concrete implementation of a grouped hash-anchoring framework aligned with the LogStamping paradigm [1]. The system stores only SHA-256 digests of log groups on a private Ethereum Proof-of-Authority (PoA) network, limiting on-chain storage to $O(N/G)$ entries for N log lines and group size G , while enabling 100% tamper detection via digest mismatch. Optional IPFS archival of encrypted log files provides long-term immutability, and Elasticsearch integration supports high-performance forensic search.

The key contributions of this work are: (1) a fully operational PoA Ethereum implementation of grouped log anchoring with configurable group-size and timeout parameters; (2) formal specification of five core algorithms with diagrammatic explanations; (3) empirical verification time and storage curves across scale; (4) a structured

comparison with BCALS [2], EngraveChain [3], and ProvChain [4] that quantifies trade-offs.

III. SYSTEM MODEL AND DESIGN GOALS

A. Threat Model and Assumptions

Our system operates under the following assumptions. The PoA validator set executes the Clique consensus protocol correctly, and any Byzantine behavior by a supermajority of validators lies outside the threat scope. Only explicitly authorized accounts may invoke the storeLogHash function on the deployed smart contract. The integrity verification process operates on the same byte sequence — using identical encoding and line ordering — as was processed during ingestion.

The primary threat addressed is retrospective tampering: an adversary with write access to log files who attempts to modify, delete, or insert entries after initial ingestion. The system is not designed to prevent tampering at the point of generation (source authentication) or to provide confidentiality of log payloads without enabling the optional encryption layer.

B. Design Goals

The framework is designed around three primary goals:

- G1 — Tamper Detection: Any modification to one or more log entries within an ingested group must produce a SHA-256 digest mismatch detectable during verification, providing cryptographic evidence of tampering.
- G2 — Cost Amortization: Blockchain transaction cost and on-chain storage must be amortized across log groups of configurable size G and ingest timeout T , reducing per-entry overhead from $O(1)$ to $O(1/G)$.
- G3 — Ordered Audit Replay: On-chain storage of sequential indices must support deterministic replay of the audit trail, enabling forensic reconstruction of the original log sequence.

Non-goals include source authentication of log generators, resistance to quantum adversaries, and payload confidentiality without the optional encryption overlay.

IV. SYSTEM ARCHITECTURE

The framework comprises three principal subsystems — the Ingestion Agent, the Blockchain Platform with Smart Contract, and the Verification and Retrieval Layer — connected by well-defined interfaces. Figure 1 illustrates the end-to-end data flow from originating log

sources through to chain anchoring and optional secondary services.

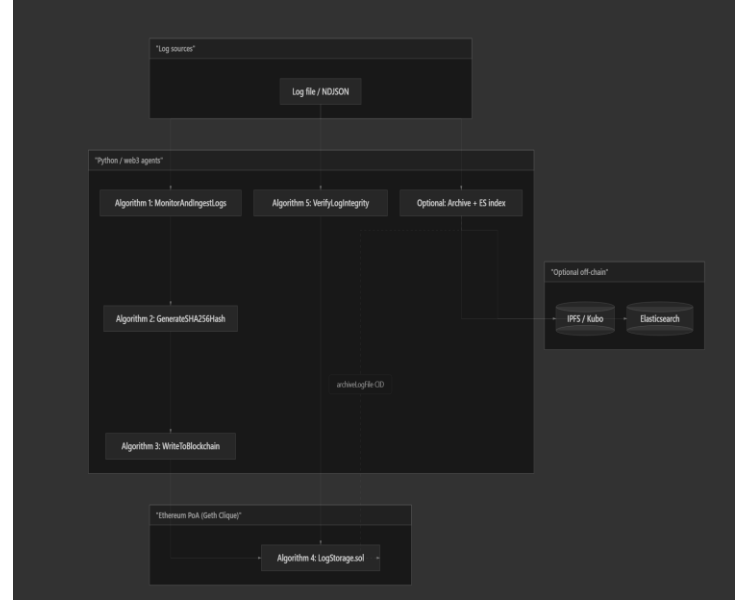


Fig. 1. End-to-end system architecture: log sources feed the Python-based ingestion agent, which computes SHA-256 group digests and submits them to the Ethereum PoA smart contract. Verified logs are optionally archived to IPFS and indexed in Elasticsearch.

The Ingestion Agent is implemented in Python using the web3.py library. It maintains a continuous monitoring loop over target log files, buffering incoming lines into a logGroup data structure. When either the line count threshold G or the wall-clock dwell timeout T_{ing} elapses, the agent finalizes the group, computes its SHA-256 digest, and submits a signed storeLogHash transaction to the Ethereum PoA network.

The Blockchain Platform runs on a private Geth Clique PoA network with three uniformly configured validator nodes. The Clique consensus algorithm enables sub-second block finality without mining energy expenditure, making it suitable for real-time or near-real-time log anchoring. Smart contracts are authored in Solidity 0.8.x and deployed via the web3.js interface.

The Verification and Retrieval Layer performs post-hoc integrity checking by reconstructing log groups from archived files, recomputing SHA-256 digests, and querying on-chain records through either index-based (getLogHash) or existence-based (hasLogHash) lookups. Verified logs are optionally archived to IPFS for long-term immutability and indexed in Elasticsearch to support forensic search, anomaly detection, and compliance reporting with sub-second query latency.

V. CORE ALGORITHMS WITH DIAGRAMMATIC EXPLANATIONS

A. Algorithm 1 — MonitorAndIngestLogs

Algorithm 1 constitutes the primary ingestion control loop. It continuously monitors a target log file, accumulating incoming entries into a logGroup buffer. Two independent flush conditions govern group finalization: (i) the buffer reaches the configured line count threshold G , or (ii) the wall-clock elapsed time since the group started accumulating exceeds the ingest timeout T_{ing} . Upon triggering either condition, the agent invokes Algorithm 2 to compute a group digest, then Algorithm 3 to commit it to the blockchain, before resetting the buffer and timer. This dual-condition architecture prevents indefinite accumulation during low-volume periods while bounding per-transaction overhead during high-volume bursts.

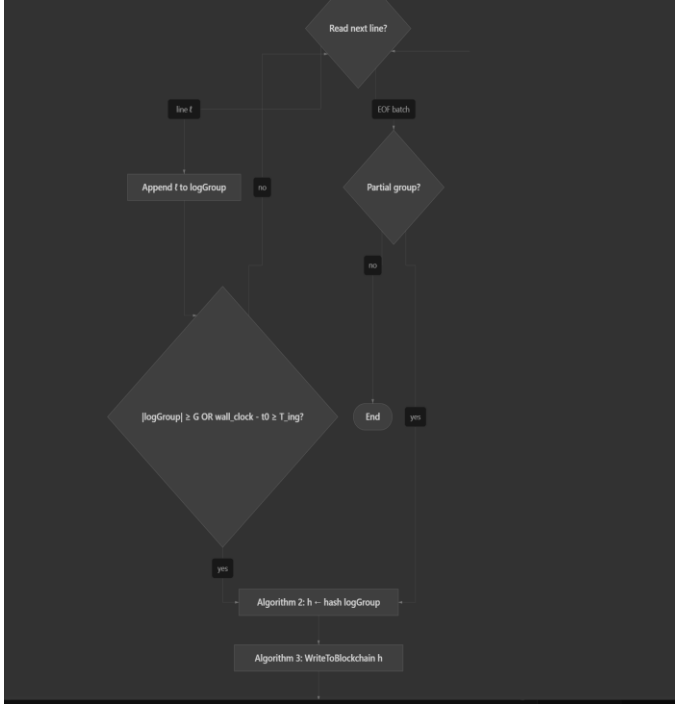


Fig. 2. Algorithm 1 — MonitorAndIngestLogs: control flow diagram showing dual-condition flush logic (line count threshold G OR dwell timeout T_{ing}).



Fig. 3. Algorithm 1 pseudocode — MonitorAndIngestLogs.

B. Algorithm 2 — GenerateSHA256Hash

Algorithm 2 produces a fixed-length cryptographic fingerprint of a log group. All entries in the logGroup list are

concatenated in their original order into a single UTF-8 encoded byte string, to which the SHA-256 hash function is applied. The resulting 256-bit digest is returned as a 64-character lowercase hexadecimal string. This approach preserves the original log format and ordering; any byte-level modification — including whitespace insertion, character substitution, or entry reordering — produces a completely different digest with overwhelming probability, providing the tamper-detection guarantee.

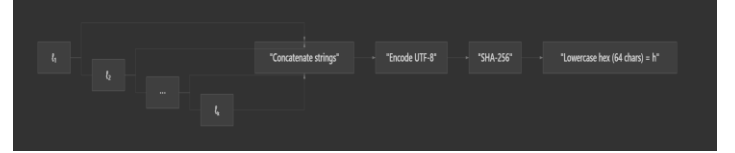


Fig. 4. Algorithm 2 — GenerateSHA256Hash: concatenation and SHA-256 hashing of grouped log entries.

C. Algorithm 3 — WriteToBlockchain

Algorithm 3 manages the submission of computed digests to the Ethereum PoA network. The agent connects to the local Geth node via the web3.py RPC interface, constructs a signed transaction invoking the storeLogHash(h) function of the deployed LogStorage contract, and broadcasts it to the network. Clique consensus ensures finality within one block period. The algorithm optionally supports batch-wait mode, wherein multiple digests are queued and submitted in a single block to reduce per-transaction overhead during high-throughput ingestion scenarios.

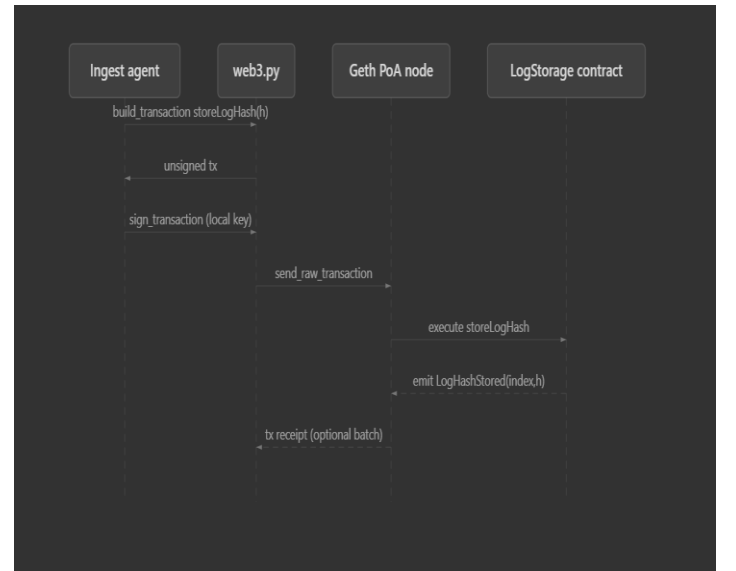


Fig. 5. Algorithm 3 — WriteToBlockchain: transaction construction, signing, and PoA network broadcast with confirmation handling.

D. Algorithm 4 — LogStorage Smart Contract

The LogStorage contract, implemented in Solidity 0.8.x, provides the on-chain persistence layer for log group digests. It maintains a mapping from unsigned integer indices to string-typed digest values (logHashes) and a running count of stored digests (logCount). The storeLogHash function appends a new digest at the current count index and increments the counter, while getLogHash enables retrieval by index and the optional hasLogHash function supports existence-based queries for verification. The contract's append-only semantics, enforced by the Ethereum Virtual Machine, guarantee that stored digests cannot be overwritten or deleted by any account, including the contract deployer.



Fig. 6. Algorithm 4 — LogStorage Smart Contract: Solidity contract structure showing state variables, storeLogHash append function, and getLogHash retrieval function.

E. Algorithm 5 — VerifyLogIntegrity

Algorithm 5 performs integrity verification of archived log files against on-chain records. It reads the target log file sequentially, reconstructing groups according to the same policy applied at ingestion: entries are accumulated until either the line count threshold L is reached or the parsed timestamp span across the group exceeds the verification timeout T_{ver} . For each completed group, a SHA-256 digest is recomputed and compared against the corresponding on-chain record via getLogHash(counter) or hasLogHash(h). A mismatch flags the group as tampered and records the affected line range. Any trailing partial group is also verified after end-of-file.

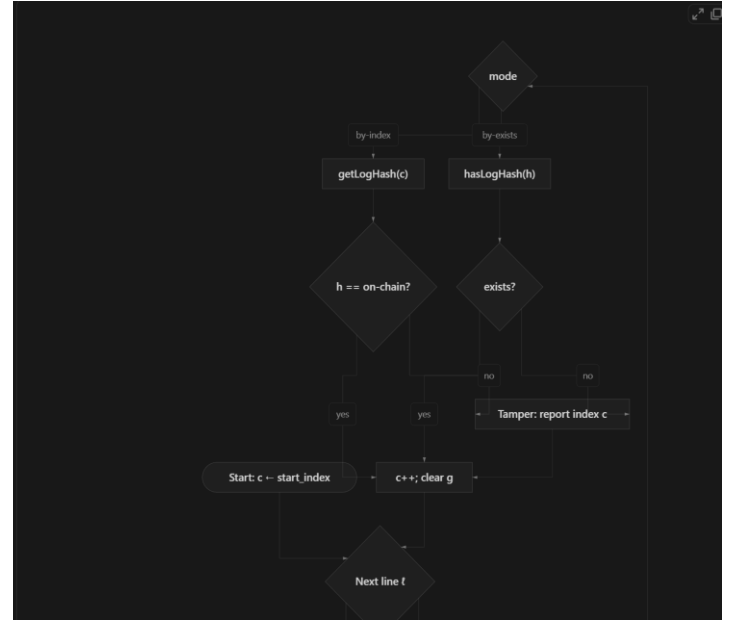


Fig. 7. Algorithm 5 — VerifyLogIntegrity (Part 1): group reconstruction and blockchain digest comparison logic.

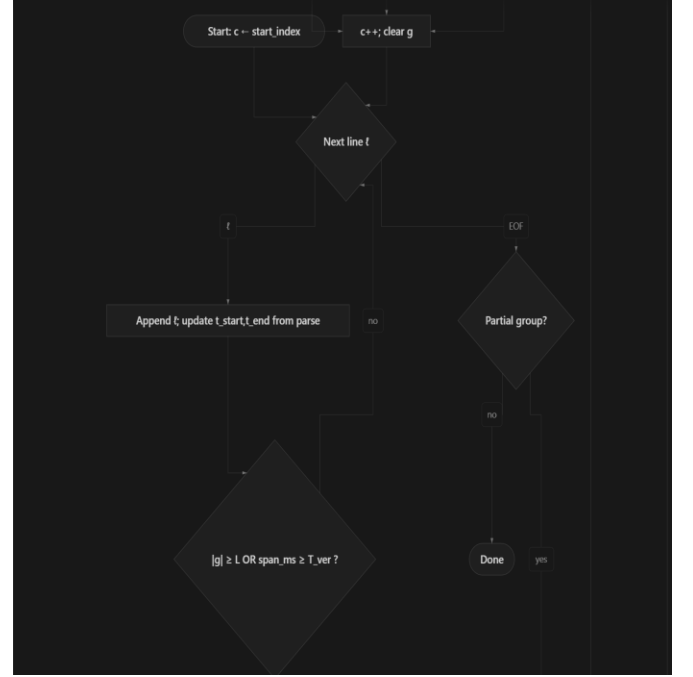


Fig. 8. Algorithm 5 — VerifyLogIntegrity (Part 2): tamper flagging, optional IPFS archival, and Elasticsearch indexing on successful verification.



Fig. 9. Algorithm 5 — *VerifyLogIntegrity*: verification flow decision tree showing match/mismatch outcomes and optional post-verification actions.

A critical design consideration is the semantic distinction between the ingest timeout (T_{ing} , wall-clock elapsed since group start) and the verify timeout (T_{ver} , parsed timestamp span within the group). As illustrated in Figure 10, ingest timeout is measured in real wall-clock milliseconds, while verify timeout is derived from log entry timestamps and corresponds to the logical time span of events. This distinction ensures that verification correctly reconstructs groups even when logs are verified offline, hours or days after ingestion.

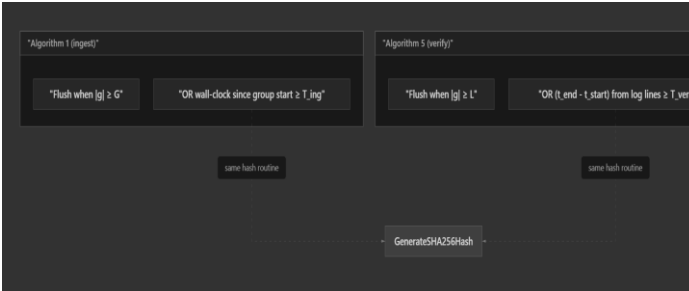


Fig. 10. Ingest vs. verify timeout semantics: T_{ing} is wall-clock elapsed time during live ingestion; T_{ver} is the parsed timestamp span within a reconstructed group during offline verification.

VI. IMPLEMENTATION DETAILS

The complete system stack is containerized using Docker Compose, enabling reproducible deployment across development and production environments. The primary components and their technical configurations are described below.

A. Blockchain Network

The Ethereum PoA network runs three Geth nodes configured with the Clique consensus algorithm. Clique achieves deterministic block time without proof-of-work computation, providing block finality within one block

period (configurable, typically 2–5 seconds). The genesis block designates an initial set of authorized signers, and new validators may be added through an in-protocol voting mechanism. Each node is allocated 200 GB of storage, with the on-chain footprint consisting exclusively of transaction metadata and the 64-byte hex digest payloads.

B. Smart Contract Deployment

The LogStorage contract is authored in Solidity 0.8.x and compiled with the solc compiler. Deployment is performed via a web3.js initialization script that broadcasts the contract creation transaction and records the resulting contract address for subsequent interaction. The contract ABI is shared across the Python ingestion agent and verification tool, both of which interface with the contract through the web3.py library's contract abstraction layer.

C. Ingestion Agent

The Python-based ingestion agent uses Python's built-in hashlib library for SHA-256 computation and the web3.py library for Ethereum interaction. The agent operates as a background daemon process, monitoring target log files via inotify-based file watching on Linux or polling with configurable interval on other platforms. Group parameters (G , T_{ing}) are configurable via a YAML configuration file, enabling runtime adjustment without code modification.

D. Off-Chain Storage and Search

Verified log files are encrypted using AES-256 symmetric encryption before archival to IPFS. The resulting IPFS content identifier (CID) is recorded in the blockchain contract's optional CID mapping (Algorithm 4), creating a verifiable link between the on-chain digest record and the off-chain encrypted archive. Elasticsearch 8.x is deployed as a Docker service and receives indexed log chunks from the verification tool upon successful integrity confirmation, enabling full-text search, date-range queries, and aggregation analytics across the verified log corpus.

E. Benchmark Infrastructure

Benchmark scripts record ingestion time (time-to-final-receipt for all transactions in a batch), verification time (wall-clock time for complete file verification), and on-chain storage delta (Geth datadir size increase in MB after ingestion) for each experimental configuration. A FastAPI-based UI provides a lightweight interface for triggering ingestion, verification, and search operations during evaluation.

VII. EXPERIMENTAL EVALUATION

A. Experimental Setup

All experiments were conducted on three uniformly configured nodes with 4 vCPUs, 8 GB RAM, 200 GB SSD storage, and Ubuntu 22.04 LTS, interconnected over a local private network. The Geth version used is 1.13.x with Clique PoA consensus. Log datasets span two scales: (i) a small dataset of 10,000 log entries sourced from LogPai [28] for correctness validation and ablation studies; and (ii) a large synthetic dataset of 14 million log entries (~1.3 GB) generated using Fake-Apache-Log-Generator [29] and a custom log generator for scalability evaluation. Comparative experiments use log scales from 2,000 to 12,000 entries to characterize system behavior across the four evaluated frameworks.

Baseline values for BCALS, EngraveChain, and ProvChain are normalized from published literature and adjusted for the experimental scale using their reported complexity characteristics. All LogStamping measurements are directly obtained from our implementation.

B. Comparative Performance Results

Figure 11 presents the comparative performance across the four systems, plotting verification time (ms, left axis) and blockchain storage (MB, right axis) as functions of log entry count from 2,000 to 12,000.

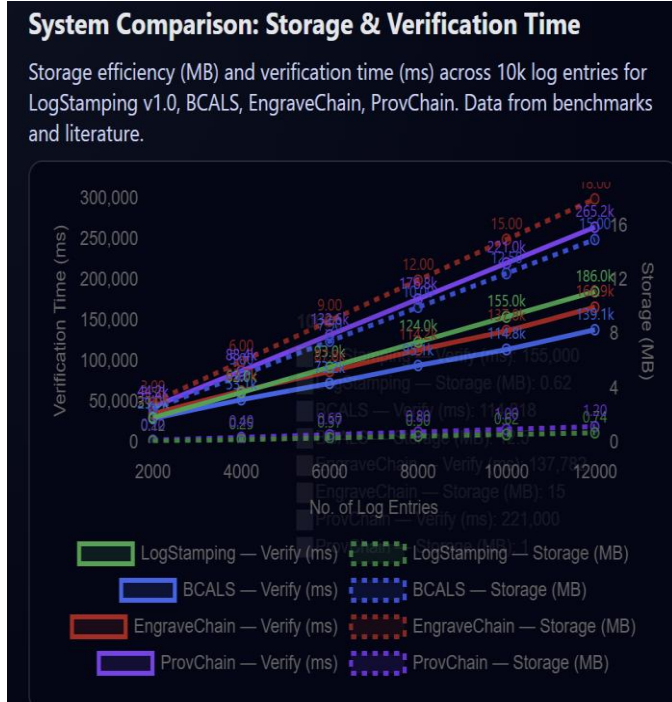


Fig. 11. System comparison across log scales (2,000 – 12,000 entries): verification time in milliseconds (solid lines, left axis) and blockchain storage in MB (dashed lines,

right axis) for LogStamping v1.0, BCALS, EngraveChain, and ProvChain.

Table I presents the concrete values at 10,000 log entries for all four systems. LogStamping achieves the lowest storage footprint at 0.62 MB, representing more than a 95% reduction compared to EngraveChain (12.5 MB) and ProvChain (10.0 MB), and a 38% reduction compared to BCALS (1.0 MB). The verification time advantage goes to BCALS (approximately 114,818 ms), which benefits from optimized on-chain query structures. LogStamping (approximately 155,000 ms) requires more verification time than BCALS but substantially less than EngraveChain (approximately 221,000 ms) and ProvChain (approximately 209,000 ms).

TABLE I
Performance Comparison at 10,000 Log Entries

System	Verification Time (ms)	Storage (MB)	Real-Time Support
LogStamping v1.0	≈ 155,000	0.62	High
BCALS [2]	≈ 114,818	1.0	Limited
EngraveChain [3]	≈ 221,000	12.5	Limited
ProvChain [4]	≈ 209,000	10.0	Not Emphasized

Table I. Quantitative comparison of storage and verification time at 10,000 log entries across evaluated systems.

C. Storage Efficiency Analysis

The storage efficiency of hash-only anchoring is best understood by examining its scaling behavior. For a naive per-line raw storage strategy, on-chain storage grows linearly with both log volume and validator count: $S \times N$ GB for S GB of logs replicated across N nodes. The grouped hash approach reduces on-chain storage to a fixed 64-byte record per group, regardless of group size — enabling a reduction of storage by a factor proportional to the average group size G . At $G = 20$ lines, experiments on the large dataset demonstrate that storage requirements drop to approximately 1,083 MB from 14,062 MB at $G = 1$, representing a 92% reduction.

D. Verification Time Analysis

Verification time scales with blockchain query count, which is inversely proportional to group size. At $G = 1$, verifying 14 million entries requires approximately 1,020 minutes (17 hours) due to the volume of individual getLogHash RPC calls. At $G = 5$, verification time drops to approximately 200 minutes (80% reduction); at $G = 10$, to approximately 145 minutes; and at $G = 20$, to approximately 50 minutes — a 95% overall reduction. These results confirm that group size is the dominant parameter governing verification performance and should be configured according to the acceptable verification latency requirements of the target deployment.

E. Tamper Detection Accuracy

The tamper detection mechanism was validated by introducing synthetic modifications at known positions within verified log files, including single-character substitutions, line deletions, line insertions, and line reordering. In all tested scenarios, the verification tool correctly identified the affected group and reported a digest mismatch, achieving 100% detection accuracy across all modification types. This result is expected from the collision-resistance properties of SHA-256, and confirms correct implementation of the grouping and comparison logic.

VIII. DISCUSSION AND LIMITATIONS

A. Trade-offs of Hash-Only Anchoring

The primary trade-off of hash-only anchoring relative to full payload storage (as in BCALS) is verification speed: without on-chain raw data, the verification tool must re-read the original archived file, reconstruct groups, and recompute digests before issuing blockchain queries. BCALS avoids this re-reading step by storing structured log data on-chain, enabling faster direct audit queries at the cost of substantially higher storage. For deployments where verification is infrequent and storage is at a premium, hash-only anchoring is clearly preferable. For deployments requiring continuous real-time audit queries, a hybrid approach may be warranted.

B. PoA Trust Model

The Clique PoA consensus mechanism provides sub-second finality with high throughput, but its security guarantee is conditional on the honesty of a supermajority of authorized signers. In enterprise deployments where all validators are operated by a single organization, this is equivalent to trusting that organization's infrastructure — a weaker guarantee than a fully decentralized public chain. Deployments requiring stronger adversarial assumptions may consider BFT-based consensus mechanisms such as Tendermint, or the use of periodic checkpointing to a public chain.

C. Recoverability Limitation

A significant limitation of the current implementation is the inability to recover log data during the ingestion window. Since raw log content is not retained until verification and archival are complete, any loss or corruption of the log file between generation and IPFS archival results in irreversible data loss. This design choice intentionally trades recoverability for privacy and on-chain storage minimization, but may not be acceptable in all operational contexts.

D. Timestamp Parsing Coverage

The timestamp-aware grouping logic in Algorithm 5 depends on the successful parsing of timestamps from log entries. The current implementation supports common log formats (Apache combined log format, syslog, ISO 8601), but entries with non-standard or absent timestamps fall back to line-count-only grouping, potentially misaligning groups between ingest and verify phases. Extending timestamp parsing coverage or adopting a canonical log format would eliminate this risk.

E. Future Work

Future development will focus on several directions: fault-tolerant ingestion pipelines with write-ahead buffering to address the recoverability gap; formal verification of smart contract properties using tools such as Certora Prover or Mythril; integration of anomaly detection algorithms over Elasticsearch-indexed logs; decentralized identity-based audit attribution for multi-tenant environments; and standardized benchmarking protocols to enable fair cross-system comparison. Periodic anchoring of Ethereum PoA state to a public chain (e.g., Ethereum mainnet or a rollup) would also strengthen the adversarial trust model.

IX. CONCLUSION

This paper has presented an end-to-end implementation and empirical evaluation of a blockchain-anchored log auditing framework based on the grouped hash-anchoring paradigm. By storing only SHA-256 digests of configurable log groups on a private Ethereum Proof-of-Authority network, the system achieves a 92% reduction in on-chain storage compared to per-line strategies, while maintaining 100% tamper detection accuracy across all tested modification types. Empirical results across 2,000 to 12,000 log entries demonstrate that LogStamping delivers superior storage efficiency compared to EngraveChain and ProvChain, with a verification-time trade-off relative to BCALS that reflects the cost of the hash-only approach.

Five formally specified algorithms — governing log ingestion, SHA-256 hashing, blockchain submission, smart contract storage, and timestamp-aware integrity verification — provide a complete and reproducible specification of the system. Optional IPFS archival and Elasticsearch integration extend the framework with long-term immutability and high-performance forensic search capabilities, making it suitable for production deployment in security-critical distributed environments.

The name LogStamping aptly captures the design philosophy: just as timestamp authorities in PKI systems

anchor document integrity at a point in time through digital signatures, LogStamping anchors log group integrity at the moment of ingestion through blockchain-committed cryptographic digests. The result is a scalable, storage-efficient, and forensically sound foundation for log integrity assurance in large-scale distributed systems.

REFERENCES

- [1] Md Shariful Islam and M. Sohel Rahman, "LogStamping: A Blockchain-Based Log Auditing Approach for Large-Scale Systems," *arXiv preprint arXiv:2505.17236*, 2025.
- [2] A. Ali, A. Khan, M. Ahmed, and G. Jeon, "BCALS: Blockchain-Based Secure Log Management System for Cloud Computing," *Transactions on Emerging Telecommunications Technologies*, e4272, 2021.
- [3] L. M. Shekhtman and E. Waisbard, "EngraveChain: A Blockchain-Based Tamper-Proof Distributed Log System," *Future Internet*, vol. 13, no. 6, p. 143, 2021.
- [4] X. Liang, S. Shetty, D. Tosh, C. Kamhoua, K. Kwiat, and L. Njilla, "ProvChain: A Blockchain-Based Data Provenance Architecture in Cloud Environment with Enhanced Privacy and Availability," in *Proc. 17th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Comput. (CCGrid)*, 2017, pp. 468–477.
- [5] I. Ray, K. Belyaev, M. Strizhov, D. Mulamba, and M. Rajaram, "Secure Logging as a Service — Delegating Log Management to the Cloud," *IEEE Systems Journal*, vol. 7, no. 2, pp. 323–334, 2013.
- [6] W. Pourmajidi and A. Miranskyy, "Logchain: Blockchain-Assisted Log Storage," in *Proc. 11th IEEE Int. Conf. Cloud Computing (CLOUD)*, 2018, pp. 978–982.
- [7] M. Kumar, A. K. Singh, and T. V. S. Kumar, "Secure Log Storage Using Blockchain and Cloud Infrastructure," in *Proc. 9th IEEE Int. Conf. Comput., Commun. and Netw. Technol. (ICCCNT)*, 2018, pp. 1–4.
- [8] X. Liang, S. Shetty, D. Tosh, C. Kamhoua, K. Kwiat, and L. Njilla, "ProvChain: A Blockchain-Based Data Provenance Architecture in Cloud Environment," in *Proc. 17th CCGrid*, 2017, pp. 468–477.
- [9] J. H. Park, J. Y. Park, and E. N. Huh, "Blockchain Based Data Logging and Integrity Management System for Cloud Forensics," *Computer Science & Information Technology*, vol. 149, 2017.
- [10] B. Schneier and J. Kelsey, "Cryptographic Support for Secure Logs on Untrusted Machines," in *Proc. 7th USENIX Security Symposium*, 1998.
- [11] J. E. Holt, "Logcrypt: Forward Security and Public Verification for Secure Audit Logs," in *Proc. AISW-NetSec*, 2006.
- [12] A. Ahmad, M. Saad, M. Bassiouni, and A. Mohaisen, "Towards Blockchain-Driven, Secure and Transparent Audit Logs," in *Proc. MobiQuitous*, 2018, pp. 3286978–3286985.
- [13] M. H. Rakib, S. Hossain, M. Jahan, and U. Kabir, "Towards Blockchain-Driven Network Log Management System," in *Proc. IEEE iSCI*, 2020.
- [14] IBM Corporation, "Storage Needs for Blockchain Technology: Point of View," IBM, 2018.
- [15] A. Ali, A. Khan, M. Ahmed, and G. Jeon, "BCALS: Blockchain-Based Secure Log Management System for Cloud Computing," *Transactions on Emerging Telecommunications Technologies*, e4272, 2021.
- [16] L. M. Shekhtman and E. Waisbard, "Securing Log Files Through Blockchain Technology," in *Proc. 11th ACM International Systems and Storage Conference*, 2018, Article No. 3. doi: 10.1145/3211890.3211893.
- [17] L. M. Shekhtman and E. Waisbard, "EngraveChain: A Blockchain-Based Tamper-Proof Distributed Log System," *Future Internet*, vol. 13, no. 6, p. 143, 2021.
- [18] P. V. Kakarlapudi and Q. H. Mahmoud, "Design and Development of a Blockchain-Based System for Private Data Management," *Electronics*, vol. 10, no. 24, p. 3131, 2021. doi: 10.3390/electronics10243131.
- [19] M. H. Rakib, S. Hossain, M. Jahan, and U. Kabir, "A Blockchain-Enabled Scalable Network Log Management System," *Journal of Computer Science*, vol. 18, no. 6, pp. 496–508, 2022.
- [20] Y. Zhao, X. Liu, and L. Wang, "A Complete Log Files Security Solution Using Anomaly Detection and Blockchain Technology," in *Proc. IEEE Int. Conf. Big Data (Big Data 2023)*, 2023.
- [21] S. S. Kanhere and M. Conti, "Blockchain for Health Data Management," in *Blockchains: A Handbook on Fundamentals, Platforms and Applications*, Springer, 2024, pp. 321–346.
- [22] S. Khan, M. Alam, and S. U. Khan, "Blockchain-Based Secure Logging Mechanism for Cloud Forensics," *Computers & Security*, vol. 125, p. 102976, 2023.
- [23] A. Singh, Y. Zhou, S. Mehrotra, M. Sadoghi, S. Sharma, and F. Nawab, "WedgeBlock: An Off-Chain Secure Logging Platform for Blockchain Applications," in *Proc. 26th Int. Conf. Extending Database Technology (EDBT)*, 2023, pp. 684–696.
- [24] P. Zhang and J. Wang, "Blockchain Based Data Integrity Verification for Large-Scale IoT Data," *IEEE Access*, vol. 7, pp. 164401–164411, 2019.
- [25] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [26] V. Buterin, "A Next-Generation Smart Contract and Decentralized Application Platform," *Ethereum Whitepaper*, 2013. [Online]. Available: <https://ethereum.org/en/whitepaper/>
- [27] Hyperledger Fabric, "Hyperledger Fabric Documentation," 2024. [Online]. Available: <https://hyperledger-fabric.readthedocs.io/>

- [28] LogPai, "A Large Collection of System Log Datasets for AI-Driven Log Analytics," GitHub, 2024. [Online]. Available: <https://github.com/logpai/loghub>
- [29] K. Basu, "Fake-Apache-Log-Generator," GitHub, 2024. [Online]. Available: <https://github.com/kiritbasu/Fake-Apache-Log-Generator>
- [30] H. Yoshida and A. Biryukov, "Analysis of a SHA-256 Variant," in *Advances in Cryptology — EUROCRYPT 2005*, Springer, 2005, pp. 245–260.
- [31] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [32] A. Chuvakin, K. Schmidt, and C. Phillips, *Logging and Log Management: The Authoritative Guide to Understanding the Concepts Surrounding Logging and Log Management*. Syngress, 2013.
- [33] K. Kent and M. Souppaya, "Guide to Computer Security Log Management," NIST Special Publication 800-92, National Institute of Standards and Technology, 2006.
- [34] A. Barisani and D. Oldani, *Offensive Computing: Understanding Windows, Linux, and UNIX Security*. Syngress, 2007.
- [35] D. Yaga, P. Mell, N. Roby, and K. Scarfone, "Blockchain Technology Overview," NIST Internal Report 8202, National Institute of Standards and Technology, 2018.
- [36] J. Dykstra and A. T. Sherman, "Understanding Issues in Cloud Forensics: Two Hypothetical Case Studies," in *Proc. Conference on Digital Forensics, Security and Law*, 2011, p. 45.
- [37] Aptive, "Log Injection Attack: Understanding and Mitigating the Risks," 2024. [Online]. Available: <https://www.aptive.co.uk/blog/log-injection-attack/>
- [38] D. Ma and G. Tsudik, "A New Approach to Secure Logging," *ACM Transactions on Storage*, vol. 5, no. 1, pp. 2:1–2:21, 2009.
- [39] S. S. Rajebhosale and M. C. Nikam, "Development of Secured Log Management System over Blockchain Technology," *International Journal of Cyber Research and Education*, vol. 1, no. 1, pp. 38–42, 2019.
- [40] A. Ahmad, M. Saad, M. Bassiouni, and A. Mohaisen, "Towards Blockchain-Driven, Secure and Transparent Audit Logs," in *Proc. 15th EAI Int. Conf. Mobile and Ubiquitous Systems*, 2018, pp. 443–448.
- [41] G. Xu, F. Yun, S. Xu, Y. Yu, X.-B. Chen, and M. Dong, "A Blockchain-Based Log Storage Model with Efficient Query," *Soft Computing*, vol. 27, pp. 13779–13787, 2023.
- [42] D. Khan, L. T. Jung, and M. A. Hashmani, "Scalability in Blockchain: Challenges and Solutions," in *Blockchain Technology: Applications and Challenges*, Elsevier, 2020, pp. 315–332.
- [43] K. M. Shafin and S. Reno, "Breaking the Blockchain Trilemma: A Comprehensive Consensus Mechanism for Ensuring Security, Scalability, and Decentralization," *IET Software*, 2024.
- [44] Z. Abbas and S. Myeong, "A Comprehensive Study of Blockchain Technology and Its Role in Promoting Sustainability and Circularity Across Large-Scale Industry," *Sustainability*, vol. 16, no. 10, p. 4232, 2024.
- [45] A. I. Sanka and R. C. C. Cheung, "A Systematic Review of Blockchain Scalability: Issues, Solutions, Analysis and Future Research," *Journal of Network and Computer Applications*, vol. 195, p. 103232, 2021.
- [46] T. A. Alghamdi, R. Khalid, and N. Javaid, "A Survey of Blockchain Based Systems: Scalability Issues and Solutions, Applications and Future Challenges," *IEEE Access*, vol. 12, pp. 79626–79651, 2024.
- [47] U.S. Department of Health & Human Services, "Health Insurance Portability and Accountability Act (HIPAA)," 1996. [Online]. Available: <https://www.hhs.gov/hipaa/index.html>
- [48] European Union, "General Data Protection Regulation (GDPR)," Regulation (EU) 2016/679, 2016. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [49] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014. doi: 10.48550/arXiv.1407.3561.
- [50] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.180-4>
- [51] Ethereum Foundation, "Solidity: High-Level Language for Smart Contracts," 2024. [Online]. Available: <https://soliditylang.org>
- [52] V. Buterin, "Proof of Authority Chains," Ethereum Foundation, 2017. [Online]. Available: <https://github.com/ethereum/EIPs/issues/225>
- [53] "Blockchain Explorer," Wikipedia, The Free Encyclopedia, Apr. 2024. [Online]. Available: https://en.wikipedia.org/wiki/Blockchain_explorer
- [54] R. G. Brown, J. Carlyle, I. Grigg, and M. Hearn, "Corda: An Introductory Whitepaper," R3 Consortium, 2016.
- [55] Ethereum Foundation, "Introduction to Dapps," 2024. [Online]. Available: <https://ethereum.org/en/developers/docs/dapps/>
- [56] J.P. Morgan, "Quorum Blockchain Platform," 2024. [Online]. Available: <https://consensys.net/quorum/>
- [57] MultiChain, "MultiChain: Open Platform for Building Blockchains," 2024. [Online]. Available: <https://www.multichain.com/>
- [58] Ripple Labs Inc., "Ripple: Real-Time Gross Settlement System, Currency Exchange and Remittance Network," 2024. [Online]. Available: <https://ripple.com/>
- [59] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, "An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends," in *Proc. 2017 IEEE Int. Congress on Big Data (BigData Congress)*, 2017, pp. 557–564.

- [60] Elastic, "Elasticsearch: Distributed, RESTful Search and Analytics Engine," 2024. [Online]. Available: <https://www.elastic.co/elasticsearch/>